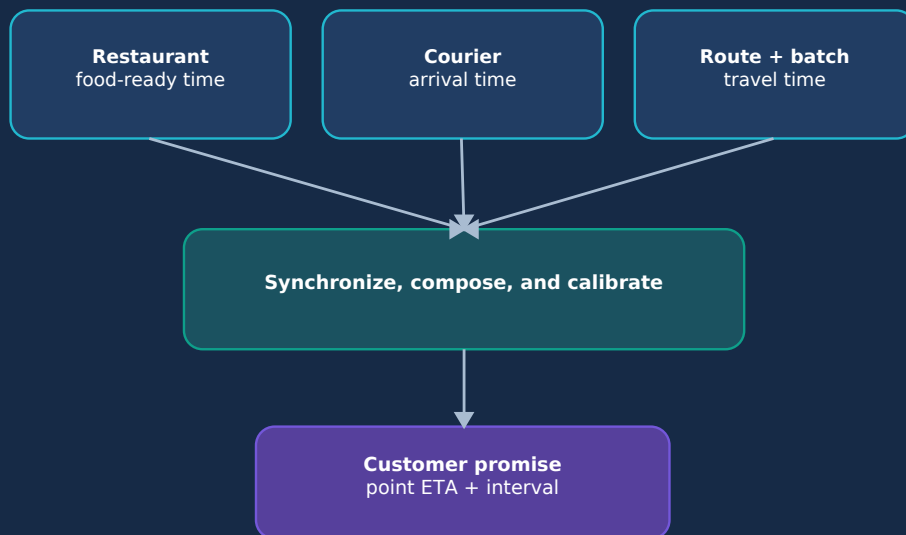


DAY 02 / 100

Design a Food Delivery ETA Prediction System

DoorDash / Swiggy-style order fulfillment prediction



Predict preparation, synchronize courier arrival, model the route, and calibrate the promise.

PUBLIC REVISION NOTES | 18-PAGE ML-FOCUSED EDITION

Define the prediction surfaces and customer promise

Food-delivery ETA is an order-fulfillment prediction problem. The available information and dominant uncertainty change as the order moves through its state machine.

Surface	Prediction endpoint	Available information	Dominant uncertainty
Checkout	Place order now -> delivered	Cart, restaurant, destination, historical supply/load	Prep + dispatch + travel
Accepted / preparing	Current time -> delivered	Accepted order, elapsed prep, kitchen signals	Food readiness + courier assignment
Courier assigned	Current time -> delivered	Courier identity/location/route	Readiness-arrival synchronization
Food ready / pickup	Current time -> delivered	Readiness known, courier/pickup state	Pickup wait + route + batching
In transit	Current time -> delivered	Current route, progress, traffic, stop sequence	Remaining travel + handoff

Business objective

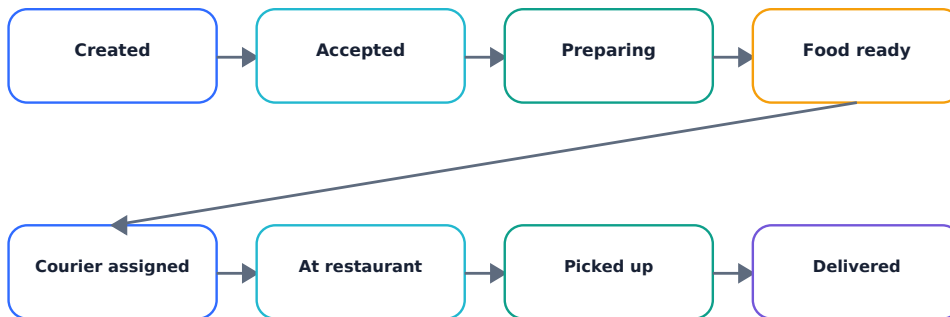
Improve promise accuracy and trust, checkout conversion, customer satisfaction, restaurant/courier coordination, dispatch and batching decisions, while reducing late deliveries, cancellations, refunds, support contacts, courier idle time, and prediction cost.

Operational ETA and customer promise are different outputs. The promise may use a calibrated quantile or interval; never expose an uncalibrated raw point prediction as a guarantee.

Primary ML task

At prediction time t , estimate remaining minutes $y_{\text{total}}(t) = t_{\text{delivered}} - t$, plus component completion times and uncertainty. Every training row must carry its surface/state because the same feature can mean something different before assignment, during preparation, or after pickup.

State machine and overlapping clocks



Preparation and courier movement overlap; total ETA is not a naive sum of every clock.

Figure 1. Restaurant work and courier dispatch/travel can proceed in parallel.

Component decomposition

Component	Clock / endpoint	Main mechanism
Food readiness	Order accepted/current time -> food ready	Menu complexity, queue, kitchen execution
Courier assignment	Eligible dispatch start -> courier assigned	Supply, matching, acceptance
Courier arrival	Assignment/current GPS -> restaurant arrival	Route, traffic, vehicle, parking/access
Pickup service	Both ready/arrived -> picked up	Queue, handoff, packaging, verification
Delivery travel	Pickup/current route point -> destination	Route, traffic, progress, stops
Batch detour	Increment caused by other pickups/drop-offs	Stop sequence and dependency
Customer handoff	Arrival nearby -> delivered	Building/access/contact friction

Do not compute total ETA as prep + courier-to-restaurant travel when those clocks overlap. Compose event times and synchronization, then add post-pickup work.

Targets, labels, and the ETA composer

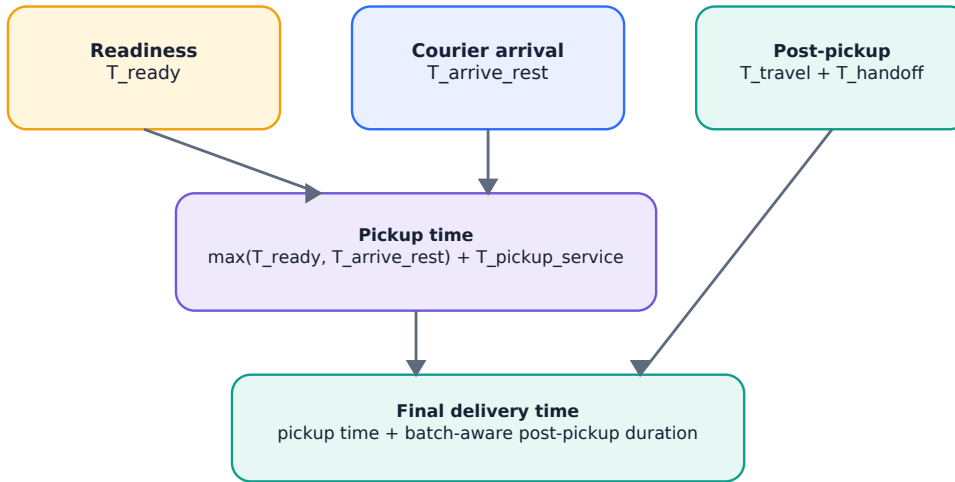


Figure 2. The $\max()$ captures the readiness-arrival dependency that a naive additive model misses.

Output	Training label at snapshot t	Notes
Ready remaining	$\max(0, t_{\text{food_ready}} - t)$	Train only where readiness is not yet observed; keep censored/invalid states separate
Assignment remaining	$\max(0, t_{\text{assigned}} - t)$	Available before assignment; market/supply label
Courier-arrival remaining	$\max(0, t_{\text{at_restaurant}} - t)$	Condition on assigned courier or candidate courier for dispatch scoring
Pickup-service time	$t_{\text{picked_up}} - \max(t_{\text{food_ready}}, t_{\text{at_restaurant}})$	Non-negative operational service after both sides are ready
Post-pickup travel	$t_{\text{arrive_customer}} - t_{\text{picked_up}}$	Route residual target can use base route ETA
Handoff time	$t_{\text{delivered}} - t_{\text{arrive_customer}}$	Use a precisely defined arrival-near-destination event
Final remaining ETA	$t_{\text{delivered}} - t$	Direct head/composer correction and customer calibration target

Travel residual

$\text{delta_travel} = \text{actual_post_pickup_travel} - \text{routing_base_eta}(\text{snapshot})$; $\text{predicted_travel} = \text{routing_base_eta} + \text{model_residual}$

Store both component and final labels. Components improve diagnosis and dispatch usefulness; the final label catches dependency and composition error.

Online architecture and responsibility boundaries

The orchestration layer reads a versioned order snapshot, assembles only features available at that time, invokes component models, composes event times, and applies promise calibration.

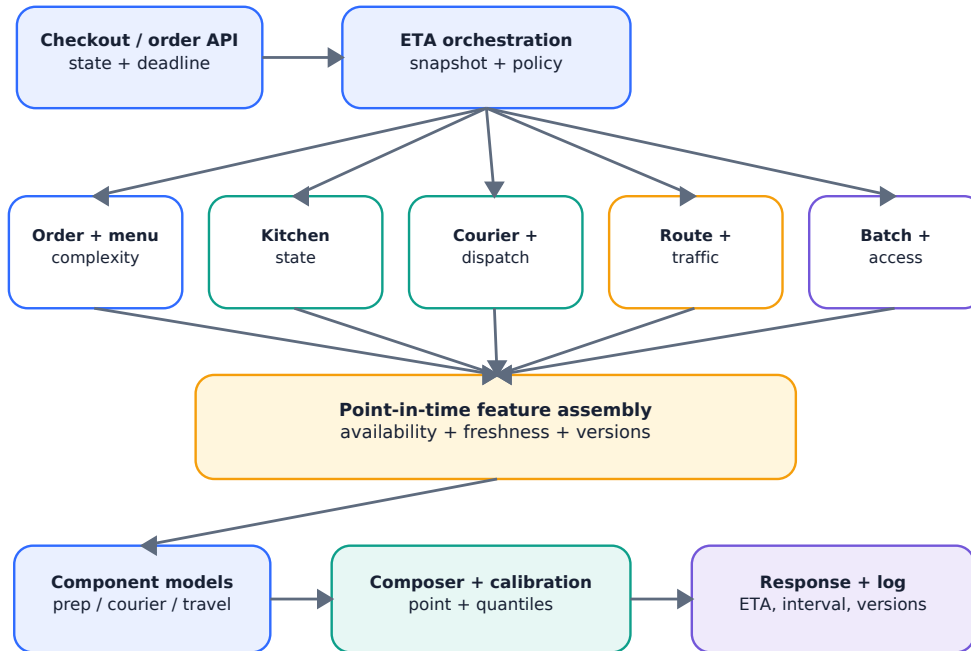


Figure 3. The online request and the delayed training snapshot share feature definitions and versions.

Boundary	Contract	Fallback
Order state	order/surface/state, timestamps, sequence/version	Reject stale transition; retain newest valid snapshot
Restaurant	metadata, queue/load/progress, freshness and quality	Restaurant/cuisine/zone historical priors
Dispatch/courier	assignment, candidate/current courier, GPS, vehicle, batch state	Supply priors or last valid courier state
Routing	route/version, base ETA, distance, segments, traffic	Cached safe route or historical travel
Model/composer	components, final point/quantiles, model versions	Reduced-feature model or deterministic composition
Response	ETA, interval, promise policy, prediction ID	Conservative bounded promise; never return impossible state

Restaurant readiness: the highest-value pre-pickup model

Feature family	Explicit examples	Transformation / reason
Restaurant identity	restaurant_id, chain, cuisine, format, kitchen type	Embedding/target-safe history; captures persistent behavior
Order complexity	item IDs/categories, quantities, modifiers, special requests, combo count	Embeddings + counts + learned item prep priors
Kitchen load	accepted/active orders, queue depth, items in progress, recent throughput	Rolling windows; strongest live regime signal
Prep progress	accepted, confirmed, started, partial-ready, elapsed since transition	State indicators + elapsed/survival-style time features
Temporal	meal period, hour, weekday, holiday, local event	Buckets/cyclic encodings; workload and staffing vary
Recent history	EWMA/median/P80/P90 prep time, recent residual, cancellation rate	Event-time rolling windows; recency and sample count
Courier coupling	predicted courier arrival, assigned state, restaurant distance	Interaction with readiness; useful for dispatch and pickup wait
Data quality	queue source age, progress source, missing flags, update count	Let model/fallback respond to stale or incomplete state

Cold start hierarchy

restaurant ID -> chain/category -> cuisine -> neighborhood/zone -> city/global prior. Blend identity-specific estimates according to sample size and recency instead of memorizing sparse restaurants.

Training caution

- The ready timestamp may be manually delayed or missing; validate against pickup and kitchen events.
- Do not use final prep duration as an online feature; at snapshot time only elapsed time and observed progress are available.
- Orders cancelled before readiness are censored, not ordinary zero-duration examples.
- Repeated snapshots from long orders can dominate training; sample or weight by order/horizon/state.

Predict a completion time or remaining duration conditioned on elapsed progress, not merely a restaurant's average prep time.

Courier assignment, arrival, and readiness synchronization

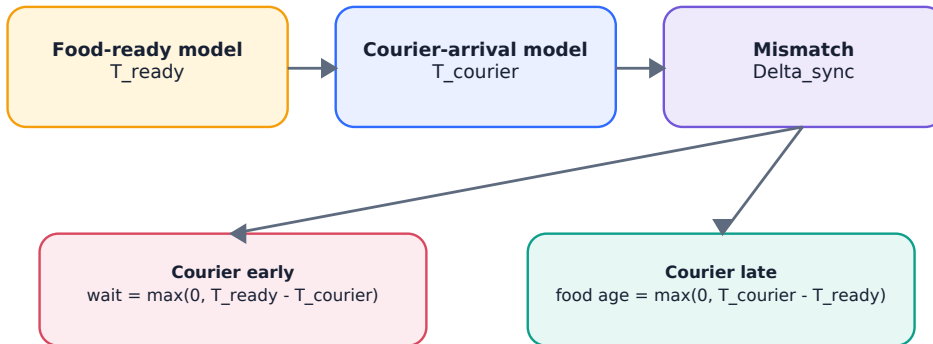


Figure 4. Early courier arrival increases wait; late arrival increases food age and customer delay.

Task	High-value features	Model output / use
Assignment delay	available couriers by radius, demand/supply ratio, acceptance history aggregate, incentives, vehicle, zone, meal peak	Time-to-assignment quantiles; checkout and pre-assignment ETA
Courier arrival	current GPS/map match, restaurant route/base ETA, traffic, vehicle, active task, parking/access history	Arrival timestamp/residual for assigned or candidate courier
Synchronization	T_ready, T_courier, their quantiles, confidence/freshness, restaurant pickup congestion	Expected courier wait, food wait, pickup start distribution
Dispatch score	customer delay, courier wait, food age, detour, acceptance/capacity constraints	Candidate-specific cost; assignment policy chooses feasible minimum

Candidate-level formulation

For each feasible courier c , predict $T_{\text{arrive}}(c)$, customer delivery quantiles, courier wait, and food-age risk. Dispatch consumes these as costs; it must not call ETA recursively after each partial choice without a versioned decision snapshot.

ETA and dispatch influence each other. Keep the model predictive and the assignment policy explicit so the objective, constraints, and feedback loop remain auditable.

Travel, pickup, batching, and handoff features

Component	Explicit features	Important interactions
Courier-to-restaurant	route/base ETA, distance, road class, turns, traffic ratio, incidents, GPS confidence, vehicle	rain x vehicle; restaurant access x peak
Pickup service	restaurant queue, food-ready flag, courier arrival gap, packaging/verification, pickup-zone congestion	readiness mismatch x restaurant; batch pickup count
Delivery travel	remaining route, progress, segment speeds, historical residual, reroute/off-route, weather	traffic x road class; route horizon x time
Batching/detour	number of orders/stops, stop sequence, other order states, incremental distance/time, route overlap	other order readiness x stop order; detour x SLA
Handoff/access	building/zone type, parking/elevator/security proxy, curb-to-door distance, contactless flag, aggregate history	time of day x building; weather x access
Freshness/quality	GPS age/accuracy, traffic age/coverage, route version, batch version, missing indicators	quality gates widen interval or trigger fallback

Three-entity geography

Represent courier, restaurant, and customer jointly using route structure plus coarse/medium/fine spatial cells. Map-match courier GPS before deriving progress. Use grid/zone embeddings for local patterns, but retain road attributes and distances so new areas back off gracefully.

Batch sequence representation

[current courier state] + [ordered remaining pickups/drop-offs] + [each order readiness/deadline] -> route/batch encoder -> per-order detour and ETA heads

Ignoring batching produces systematic underprediction. A batch is an ordered, changing route plan, not a single binary feature.

Feature transformations and state availability

Transformation	Use	Leakage / serving rule
ID embeddings / hashing	restaurant, item/category, zone/grid, road segment	Unknown bucket + hierarchy; bound vocabulary and lookup latency
Normalized raw + buckets	distance, queue, speed, elapsed time, item count	Fit transforms on training period; preserve extremes/missingness
Rolling/EWMA aggregates	recent prep, throughput, wait, traffic residual	Event-time windows only; include count, age, and fallback level
Ratios/residuals	current vs historical speed/prep; route residual	Align to the same route/snapshot and avoid division near zero
Interactions	load x order size, rain x traffic, arrival x readiness	Prefer explicit high-value interactions for GBDT; deep model can learn more
Freshness/quality	source age, coverage, confidence, missing flags	Feature contract decides drop, default, interval widening, or fallback

Feature group	Checkout	Preparing	Assigned	In transit
Restaurant/menu history	Yes	Yes	Yes	Low value
Live kitchen queue/progress	Maybe	Yes	Yes	Observed/complete
Specific courier GPS/route	No	Maybe	Yes	Yes
Dispatch/supply state	Yes	Yes	Observed	Low value
Batch stop sequence	Candidate/unknown	Candidate	Yes	Yes
Remaining route/progress	Estimate	Estimate	Estimate	Yes
Customer access history	Yes	Yes	Yes	Yes

A model must not silently default a not-yet-available feature to the same value as a missing or failed feature. Encode availability, missingness, and freshness separately.

Recommended baseline: specialized GBDTs plus a composer

Start with separate LightGBM/XGBoost models for mechanisms that have different labels and feature availability. Compose timestamps, then fit a lightweight final correction/calibration model.

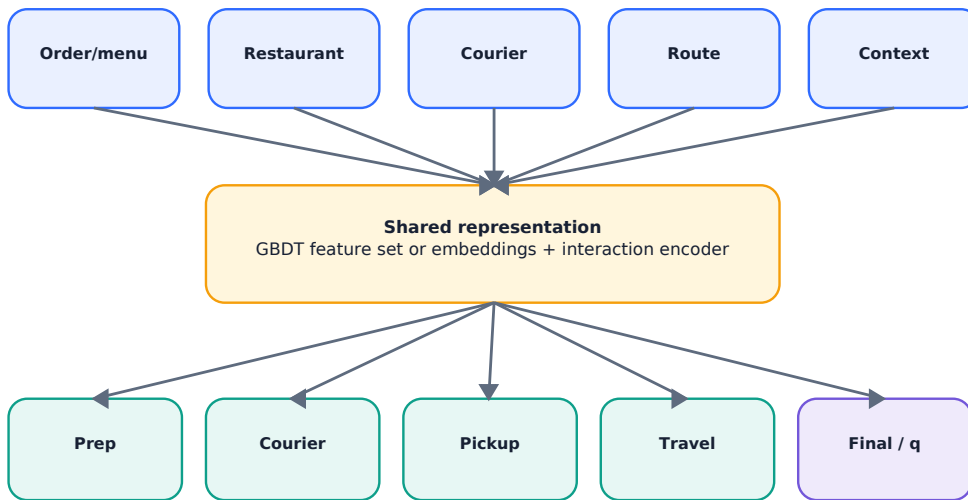
Model	Inputs	Output	Why separate initially
Prep readiness	order + restaurant + kitchen + elapsed progress	ready remaining or ready-time quantiles	Dominant pre-pickup mechanism; direct diagnostics
Assignment	supply/demand + zone + peak + order context	assignment-delay quantiles	Only exists before courier assignment
Courier arrival	specific courier + map match + route + traffic	arrival residual/quantiles	Routing-grounded and candidate-specific
Pickup service	readiness gap + queue + restaurant + batch	service-time quantiles	Captures restaurant handoff after clocks meet
Delivery travel	route + traffic + progress + vehicle + weather	routing residual/quantiles	Reuses strong ride-hailing formulation [1]
Handoff	customer zone/building + access + time/weather	handoff duration	Distinct curb-to-door behavior
Composer correction	all component predictions + state + uncertainty	final residual/quantiles	Learns dependency and systematic composition error

Why GBDT first

- Excellent tabular nonlinear baseline with missing values and high-value engineered interactions.
- Fast inference, feature attribution, segment debugging, and easier retraining/rollback.
- Establishes whether embeddings or sequence models add value beyond clean features and labels.

The baseline is not one monolithic total-ETA regressor. It is a causal-ish operational decomposition with a final consistency layer.

Advanced model: shared representation with semantic heads



Shared learning helps sparse restaurants; semantic heads keep component errors diagnosable.

Figure 5. A shared model reduces fragmented infrastructure while preserving component supervision.

Upgrade	When it helps	Guardrail
Restaurant/item/geo embeddings	High-cardinality entities, sparse interactions, cold-start sharing	Memory, unseen IDs, hierarchy/backoff
Multi-task shared encoder	Related components share order, restaurant, time, and geography	Avoid negative transfer; state-mask unavailable heads
Feature-token attention	Complex cross-feature interactions exceed engineered GBDT gains	Bound token count; measure P99 and real kernels
Sequence/route encoder	Ordered batch stops, progress events, or route segments matter	Version/length control; avoid stale sequences
Mixture/segment adapters	Cities/restaurants have distinct regimes but need global sharing	Regularize and monitor fragmentation

Head masking and consistency

At each state, compute loss only for valid targets. A pre-assignment snapshot has an assignment head but no specific-courier arrival head. Add final-ETA supervision to keep components jointly useful; enforce non-negative durations or monotonic quantiles through parameterization/post-processing.

Move beyond GBDT only after point-in-time labels, component baselines, latency budgets, and segment evaluation are reliable. DeepETA is useful inspiration for embeddings, attention, residual learning, and low-latency modeling [1].

Loss functions and multi-task training objective

Loss	Formula / behavior	Use
MAE	$ y - y_{\text{hat}} $	Direct minutes error; robust but non-smooth
Huber	$0.5e^2$ near zero; $\delta(e -0.5\delta)$ in tail	Stable default under kitchen/traffic outliers
Asymmetric Huber	$w_{\text{under}} H(e)$ if $y_{\text{hat}} < y$; $w_{\text{over}} H(e)$ otherwise	Penalize missed promises differently from pessimism
Pinball / quantile	$\max(qe, (q-1)e)$, $e=y-y_{\text{hat}_q}$	Learn P50/P80/P90 and customer promise uncertainty
Censored likelihood	Probability of event occurring after censor time	Optional advanced treatment for unresolved readiness/delivery

Multi-task objective

$L_{\text{total}} = \sum_k \lambda_k * m_k * L_k(y_k, y_{\text{hat}_k}) + \lambda_{\text{final}} * L_{\text{final}} + \lambda_q * L_{\text{quantiles}} + \lambda_{\text{cons}} * L_{\text{consistency}}$

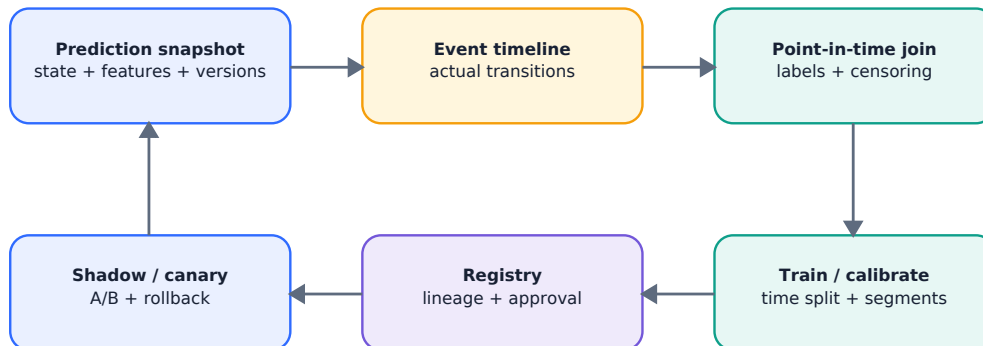
- m_k masks a component head when its label is not valid for the snapshot state.
- λ values reflect label scale, uncertainty, product importance, and gradient balance; tune on validation and business cost.
- $L_{\text{consistency}}$ can penalize impossible negative durations, quantile crossing, or final predictions inconsistent with component event times.
- Weight repeated snapshots so a single long-running order does not dominate the objective.

Point prediction versus promise

Use P50 or an expected/robust point estimate for operational planning; use a calibrated P80/P90-style value or interval for a reliability-oriented customer promise. Choose the quantile through experiment, not by convention.

Loss is product policy in mathematical form. Underprediction, excessive buffer, courier wait, food age, and final lateness do not have identical costs.

Point-in-time dataset, delayed labels, and validation



Completed orders reveal endpoints later; cancellations and substitutions need explicit handling.

Figure 6. Prediction snapshots are joined to the final event timeline only after labels mature.

Snapshot schema

prediction_id, order_id, prediction_time, surface/state, state_version, restaurant_version, courier_id_if_known, route_version, batch_version, component_predictions, final_point/quantiles, model_versions, feature values/references, source timestamps, freshness/quality, raw_and_displayed_promise

Dataset rules

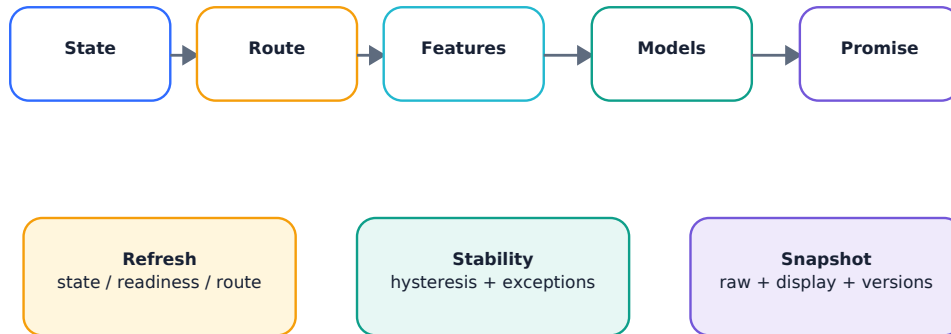
- Join features as-of prediction time; later queue, traffic, route, or batch revisions are leakage.
- Split by time, then test unseen restaurants/zones and major regime changes separately.
- Retain cancellations, substitutions, restaurant closures, and failed deliveries with reason/censoring status.
- Deduplicate retries and validate monotonic event versions before deriving component timestamps.
- Sample meaningful snapshots around state changes plus controlled intervals; avoid every GPS heartbeat.
- Train calibrators on held-out recent data, separate from model fitting, and refresh after drift.

Rollout

Offline replay -> shadow predictions -> small canary -> A/B test -> segment expansion. Compare against historical/routing/composed baselines and retain one-click rollback to the last known-good model/policy.

Michelangelo illustrates the reusable platform loop of data management, training, evaluation, deployment, prediction, and monitoring [2]. The food-specific labels and state contracts remain the application's responsibility.

Event-driven refresh, caching, and displayed stability



Recompute when information changes; do not spend inference on unchanged GPS heartbeats.

Figure 7. Prediction freshness and customer-visible stability are related but separate policies.

Trigger	Example	Action
Order/restaurant state	accepted, progress, food ready, closure	Recompute immediately; change valid heads
Assignment/batch	courier assigned/reassigned, stop sequence changed	Invalidate courier/batch features and route
Courier progress	meaningful distance/time threshold, off-route	Map-match, update route progress, recompute
Traffic/weather	material corridor or weather regime change	Refresh affected routes/orders
Maximum age	no event for configured horizon	Safety refresh and freshness recheck

Low-latency tactics

- Precompute restaurant/menu/history aggregates.
- Keep live kitchen/order/courier state in low-latency storage.
- Cache stable metadata and route results with versions.
- Use sparse embedding lookup and small serving heads.
- Assemble only state-relevant features.
- Deadline-aware reduced-feature fallback.
- Event-driven inference, not every heartbeat.
- Track routing/features/model/calibration latency separately.

Displayed stability

Use small-change hysteresis or confidence-aware smoothing, but bypass it after food-ready, reassignment, reroute, batch change, or a material incident. Log raw and displayed ETA; measure volatility, maximum jump, and delay in surfacing genuine change.

Metrics, calibration, and required slices

Layer	Metrics	Question
Point accuracy	MAE, Median AE, RMSE	How large is typical and squared error?
Tail/bias	P90/P95 AE, signed bias, underprediction rate	Where are promises dangerously optimistic?
Components	prep/assignment/arrival/pickup/travel/handoff MAE	Which mechanism is responsible?
Uncertainty	P50/P80/P90 coverage, interval width, crossing	Do quantiles mean what they claim?
Stability	update volatility, max jump, correction delay	Does ETA change sensibly over time?
Business	conversion, cancellation, late/refund/support, satisfaction	Does the promise improve outcomes?
Operations	courier wait/utilization, food age, batch efficiency	Does coordination improve without hidden cost?
System	P50/P95/P99 latency, errors, fallback, cost	Can it run reliably at peak?

Required slices

- Checkout vs preparing vs assigned vs in transit.
- New vs mature restaurant; cuisine/chain/zone.
- Small vs complex order; low vs high kitchen load.
- Single vs batched; stop position and detour size.
- City/market, meal peak, holiday, weather.
- Fresh vs stale/missing kitchen, GPS, traffic.
- Short vs long horizon; vehicle and route class.
- Completed vs cancellation/censoring segments.

Calibration

A displayed P80 should achieve about 80% empirical on-time coverage in the evaluated population. Calibrate by surface and major regime with enough data; monitor both coverage and interval width so reliability is not achieved through unlimited padding.

Do not optimize only MAE. A lower global MAE can coexist with worse missed promises, food age, courier wait, batch outcomes, or critical restaurant segments.

Monitoring, failure modes, and trade-offs

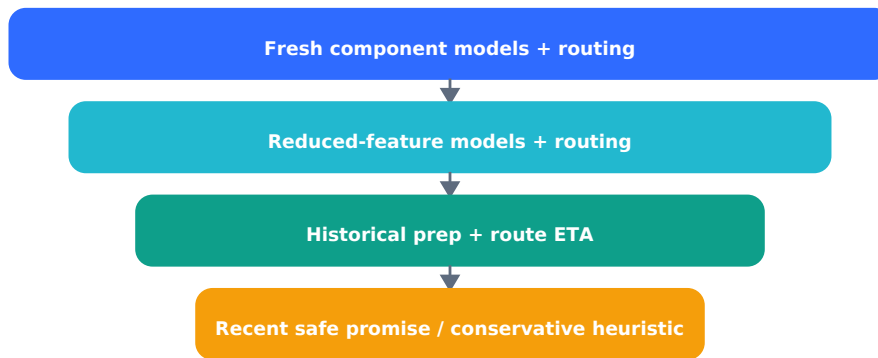


Figure 8. Degrade component quality stepwise while keeping the order state and promise coherent.

Failure / drift	Detection	Response
Kitchen state stale	source age, missing updates, queue anomaly	Historical restaurant/cuisine prep prior + wider interval
Courier GPS/map match weak	accuracy, jump, confidence, off-route	Last valid point, route fallback, uncertainty increase
Traffic/routing unhealthy	age, coverage, latency, failures	Historical traffic or cached safe route
Batch state inconsistent	version regression, invalid stop/order	Use newest valid dispatch snapshot; reject stale update
Model/calibration unhealthy	error/P99/fallback/coverage canary	Last-known-good or deterministic composer
Segment drift	prep/travel bias and tail by slice	Rollback, recalibrate, retrain, or segment safeguard

Decision	Option A	Option B	Rule
Model	Single end-to-end	Components + composer	Prefer components for mechanisms/diagnosis; final head corrects composition
Prep	Per-restaurant	Shared + embedding	Share globally; specialize only with enough fresh data
Output	Point	Quantiles	Use quantiles only with calibrated product policy
Refresh	Fixed timer	Events/thresholds	Event-driven at scale; keep maximum-age safety
Batching	Ignore	Explicit sequence	Sequence is required when detour materially affects ETA

Mental model and interview checklist

Step	Question
SURFACE	Checkout, preparing, assigned, pickup, or in-transit ETA?
LABEL	Which exact event timestamps define final and component targets?
STATE	Which restaurant, courier, dispatch, route, and batch versions are valid now?
READINESS	What order/kitchen/history signals predict food-ready time?
COURIER	What predicts assignment and candidate/specific courier arrival?
SYNC	How do readiness and arrival distributions determine pickup wait and food age?
ROUTE/BATCH	What does routing provide, what residual does ML learn, and how do other stops alter ETA?
MODEL/LOSS	Why component GBDTs or multi-task heads; why Huber, asymmetry, and quantiles?
TRAIN	Are joins point-in-time correct, cancellations explicit, and splits temporal/cold-start aware?
SERVE	What triggers refresh; what are the P99, stability, and fallback policies?
MEASURE	Are point, tail, bias, coverage, component, business, operational, and system metrics covered?

Food delivery versus ride-hailing ETA

Ride-hailing emphasis	Food-delivery addition
Routing and traffic dominate after assignment	Restaurant readiness can dominate before pickup
Driver and rider endpoints	Courier, restaurant, customer, plus ordered batch stops
Mostly sequential travel	Preparation and courier movement overlap and must synchronize
Routing + residual travel model	Reuse travel residual, then add readiness, pickup, batching, and handoff
Point ETA and stability	Quantile promise, food age, courier wait, and component calibration

One-line takeaway: predict when food and courier will both be ready, then model the batch-aware route and handoff, while continuously recalibrating a customer promise from point-in-time state.

Approved references and public-use note

[1] Uber Engineering. **DeepETA: How Uber Predicts Arrival Times Using Deep Learning**. February 10, 2022.
uber.com/blog/deepeta-how-uber-predicts-arrival-times

[2] Uber Engineering. **Meet Michelangelo: Uber's Machine Learning Platform**. September 5, 2017.
uber.com/in/en/blog/michelangelo-machine-learning-platform

Reference policy: these are the only external references carried forward from the prior Day 2 work. They support the travel-residual and reusable ML-platform patterns; the food-delivery decomposition is an educational design synthesis.

Public scope

This document is a DoorDash/Swiggy-style educational system design, not a description of either company's current internal implementation. Thresholds, model stages, feature contracts, fallbacks, and objective weights are proposed design choices that require validation on the actual product and data.

What was refined

The original 15-page draft was preserved in substance and reorganized around ML formulation. Component event labels, overlapping clocks, state-dependent feature availability, restaurant cold start, candidate-courier scoring, batch sequences, explicit feature transformations, head masking, multi-task loss, snapshot weighting, censoring, temporal validation, calibration, and operational fallbacks were made concrete. Repetition and truncated tables were removed.

END OF DAY 02

**Predict readiness. Synchronize the courier.
Model the route. Calibrate the promise.**